INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA

DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE
ENERO - JUNIO 2026

CARRERA
Ingeniería en Sistemas computacionales

MATERIA
Patrones de Diseño

TÍTULO
Singleton

NOMBRE Y NÚMERO DE CONTROL DEL ALUMNO
Nolasco Vazquez José Antonio - 22210326

NOMBRE DEL MAESTRO
MARIBEL GUERRERO LUIS

FECHA DE ENTREGA
09 de Marzo del 2026

Capturas de: Más Operadores

```
Operador op1 = new Operador(1, "Laura");
Operador op2 = new Operador(2, "Carlos");
//Nuevo operador
Operador op3 = new Operador(3, "Tiana");
Operador op4 = new Operador(4, "Nyra");
```

Captura de: Nuevas llamadas

```
Central_911 Llamada1 = Central_911.Obtener_Instancia();
Central_911 Llamada2 = Central_911.Obtener_Instancia();
//nueva llamada
Central_911 Llamada3 = Central_911.Obtener_Instancia();
Central_911 Llamada4 = Central_911.Obtener_Instancia();
```

```
                //Emergencias nuevas
            case "Robo":
                Console.WriteLine("Enviando patrullas");
                break;

            case "Ataque Cardiaco":
                Console.WriteLine("Enviando ambulancia");
                break;

            case "Persona atrapada":
                Console.WriteLine("Enviando equipo de rescate");
                break;

            case "Derrumbe":
                Console.WriteLine("Enviando Equipo de busqueda y rescate");
                break;
        }
```

Captura de: Verificar que es una única instancia

```
Console.WriteLine("\nVerificando si todas las llamadas usan la misma instancia:");

if (ReferenceEquals(Llamada1, Llamada2) &&
    ReferenceEquals(Llamada1, Llamada3) &&
    ReferenceEquals(Llamada1, Llamada4))
{
    Console.WriteLine("Todas las llamadas pertenecen a la misma instancia de Central_911.");
}
else
{
    Console.WriteLine("Existen múltiples instancias de Central_911.");
}
```

Identificar

1. El atributo estático de la instancia

```
private static Central_911 _instance;
```

Este es el atributo estático ya que guarda la única instancia de la clase inicial que denominamos Central_911, este pertenece a la clase y no a sus objetos haciendo que utilice sea la única instancia en el programa.

2. El constructor

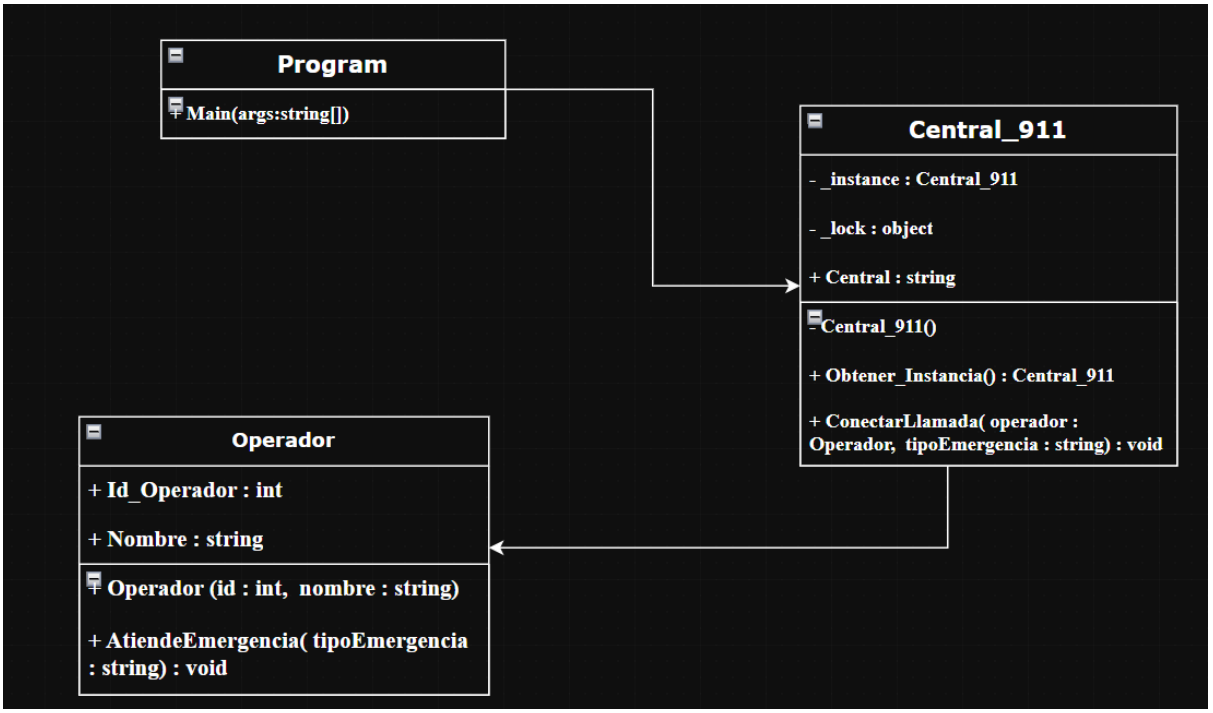
```
1 referencia
private Central_911()
{
    Central = "Central 911";

}
```

La función del constructor dentro del programa evita que se creen varios objetos de la clase y hace que el método Obtener_Instancia realice una única instancia cumpliendo el funcionamiento del Singleton

¿Por qué no se pueden crear múltiples objetos?

Esto de acuerdo a nuestro constructor ya declarado es privada lo que evita que se realicen otras instancias con NEW.

Diagrama UML



Elaborado en Draw.Io

Conclusión

Singleton hace ver que solo se necesita de una sola instancia para almacenar todo dentro de ello, es importante porque en un sistema real de emergencias debe existir una única central que administre las llamadas y coordine a los operadores. Gracias al método Obtener_Instancia(), el programa evita que se creen múltiples centrales y asegura que todos los operadores y llamadas se gestionan desde un mismo punto.